Reversible Modular Chunked Diffusion: An Architectural Path to Memory-Frugal Training and Inference

Anonymous Author(s)
Affiliation
Address
email

Abstract

Diffusion backbones deliver state-of-the-art quality but remain memory-hungry, especially during training where activation storage dominates and scales with resolution and sequence length. Most prior efficiency methods focus on inference and treat the network as a fixed graph, limiting their impact on training-time memory Zhan et al. [2024], Shang et al. [2022], Fang et al. [2023], Li et al. [2023]. We introduce Reversible Modular Chunked Diffusion (RMCD), an architectural redesign that targets memory at its source. RMCD couples four ideas: reversible dual-stream blocks to avoid saving activations; operator-aligned micro-chunking to process tensors in cache-sized tiles during forward and backward; dynamic spatial gating that adaptively skips tiles across timesteps; and low-rank parameter packs that keep base weights 8-bit while training only a small low-rank delta. We validate RMCD with a reproducible PyTorch scaffold under matched protocols. In quick functional runs on synthetic data, RMCD reduces peak training memory by 37.6% versus a UNet-like baseline and cuts inference memory by up to about 55% at 256×256. Ablations show that disabling chunking sacrifices memory gains; prototype overheads without fused kernels slow inference, clarifying engineering priorities. We release end-to-end protocols, logs, and figures to facilitate replication and to motivate architectural co-design for memory-efficient diffusion.

1 Introduction

Diffusion models have become the dominant approach for image and video generation due to their sample quality and controllability Dhariwal and Nichol [2021]. Yet their memory footprint poses a practical barrier. Even with gradient checkpointing, training modern UNet- or DiT-style backbones at high resolution retains tens of gigabytes of activations, while video models suffer from memory that scales with the product of height, width, and time. This burden often pushes fine-tuning and even modest-scale training to multi-GPU rigs.

Prior efficiency efforts are rich but largely skewed toward inference. Streamlined Inference reduces memory at sampling time by partitioning features, grouping operators, and exploiting similarity across denoising steps Zhan et al. [2024]. Post-training quantization compresses weights without retraining Shang et al. [2022], and pruning removes redundant structure Fang et al. [2023]. Architectures that emphasize FLOP efficiency, such as state-space model backbones that avoid global attention Yan et al. [2023], and mobile-oriented designs combined with step distillation also accelerate sampling Li et al. [2023]. Multistep distillation reduces the number of required denoising steps Salimans et al. [2024]. However, these advances leave a fundamental question open: can we reduce memory at its architectural source so that both training and inference benefit, without sacrificing quality?

Submitted to 1st Open Conference on AI Agents for Science (agents4science 2025). Do not distribute.

Designing such a backbone is challenging. First, eliminating activation storage reliably requires invertible computation or exact recomputation that remains numerically stable across resolutions. Second, chunking must be intrinsic to operators so that backpropagation follows the same tiling schedule; external inference-time slicing cannot influence training-time autograd state Zhan et al. [2024]. Third, any adaptive sparsification must respect diffusion dynamics: early timesteps are empirically more tolerant to approximations than later ones, which demand detail preservation aut. Finally, fine-tuning should avoid re-inflating optimizer and gradient memory; updates should be low rank and compatible with quantized bases Shang et al. [2022].

We propose Reversible Modular Chunked Diffusion (RMCD), a diffusion backbone that pursues memory efficiency as a first-class architectural objective. RMCD integrates four mechanisms. Reversible dual-stream blocks implement additive coupling so that activations can be recomputed rather than stored. Operator-aligned micro-chunking expresses core operators as cache-sized tiles and binds forward and backward to the same schedule. Dynamic spatial gating inserts a small, timestep-aware gate before each block to mask tiles at early steps and densify later, reducing both FLOPs and live memory in proportion to sparsity aut. Low-rank parameter packs represent every weight as an 8-bit base plus a trainable low-rank delta, lowering optimizer and gradient memory during fine-tuning while remaining compatible with post-training quantization Shang et al. [2022] and pruning Fang et al. [2023]. RMCD complements alternative efficiency directions including SSM backbones Yan et al. [2023], mobile-oriented architectures Li et al. [2023], and step reduction via distillation Salimans et al. [2024].

We verify RMCD using an open PyTorch scaffold that standardizes seeds, samplers, warmup, and logging. In quick functional runs on synthetic data, RMCD reduces peak training memory by 37.6% versus a UNet-like baseline and reduces inference memory by up to about 55% at 256×256. The prototype's Python-level chunking and non-fused gates slow inference; moreover, its "reversible" path lacks true reversible backpropagation, underestimating reversibility's benefit. We present ablations across reversibility, chunking, and gating that isolate each component's impact on memory and throughput.

1.1 Contributions

- **Intrinsic memory-efficient backbone:** An intrinsically memory-efficient diffusion backbone, RMCD, that targets activation memory via reversible computation, operator-aligned micro-chunking, and timestep-aware dynamic spatial gating, with low-rank parameter packs for memory-frugal fine-tuning.
- **Ablation-ready API:** A diffusers-compatible API that exposes toggles and profiling hooks for masks and chunk schedules, enabling rigorous ablations.
- **Reproducible scaffold and findings:** A reproducible evaluation scaffold and functional results demonstrating substantial memory reductions during training and inference, together with transparent documentation of prototype limitations and engineering priorities.

Looking ahead, RMCD opens a design space that elevates intrinsic memory footprint alongside sample quality and speed. It invites kernel-level co-design with Triton and CUDA graphs, integration with multistep distillation Salimans et al. [2024], semi-parametric conditioning Blattmann et al. [2022], and evaluation protocols that report peak memory jointly with FID and CLIPScore Dhariwal and Nichol [2021], Hessel et al. [2021].

2 Related Work

Inference-time memory and speed. Streamlined Inference reduces video diffusion memory by slicing features, grouping operators, and reusing computation across adjacent timesteps without retraining Zhan et al. [2024]. This approach treats the model as a static graph and primarily benefits inference. RMCD differs by embedding chunking into operator definitions themselves so that backward inherits the tiling schedule, directly lowering activation memory during training and inference.

Weight compression and pruning. Post-training quantization compresses diffusion weights to 8 bits and can even improve performance without retraining Shang et al. [2022]. Structural pruning removes redundant structure and reduces FLOPs with limited or no retraining Fang et al. [2023]. These techniques target parameter memory and inference compute. RMCD is complementary: it primarily

reduces activation memory via reversibility and chunking. Its low-rank parameter packs borrow the PTQ spirit by keeping base weights quantized while updating only a small low-rank delta, thereby reducing optimizer state during fine-tuning.

Architectural efficiency and latency. Diffusion state-space models eschew global attention and scale well to high resolutions with FLOP-efficient designs Yan et al. [2023]. SnapFusion identifies decoder redundancy and applies step distillation for mobile latency Li et al. [2023]. Multistep distillation accelerates sampling by matching conditional moments, cutting the number of denoising steps Salimans et al. [2024]. RMCD complements these lines by emphasizing invertibility and operator-intrinsic tiling to attack activation memory rather than step count or FLOPs alone.

Personalization and on-device adaptation. Hollowed Net temporarily removes deep layers during subject personalization to meet device budgets, then reinserts them for inference Cho et al. [2024]. RMCD instead preserves full depth and minimizes trainable and optimizer memory via low-rank parameter packs with quantized bases during adaptation.

Quality and evaluation. Dhariwal and Nichol established robust diffusion baselines and popularized FID reporting Dhariwal and Nichol [2021]. For video, latent diffusion strategies align temporal and spatial dimensions to scale training and evaluation Blattmann et al. [2023]. CLIPScore offers a reference-free image-text compatibility metric Hessel et al. [2021]. Our quick functional run does not compute these metrics; nevertheless, our scaffold supports them for full-scale evaluations.

Temporal robustness across steps. Multiple works observe that early denoising steps are more tolerant to approximations, motivating dynamic strategies that densify later aut. RMCD operationalizes this insight through dynamic spatial gating.

In summary, compared to inference-only slicing Zhan et al. [2024], RMCD integrates chunking into both passes; compared to PTQ and pruning Shang et al. [2022], Fang et al. [2023], RMCD focuses on activation memory; compared to SSM backbones Yan et al. [2023], RMCD emphasizes reversibility and timestep-aware spatial sparsity; and compared to layer removal during personalization Cho et al. [2024], RMCD reduces optimizer state via low-rank updates while preserving full-depth inference.

3 Background

Problem setting and sources of memory. We consider conditional diffusion models that predict a noise residual from a noisy input at a given timestep and conditioning signal. For images, tensors have shape (batch, channels, height, width); for videos, an additional temporal dimension yields (batch, channels, time, height, width). Peak memory during training arises from parameters and optimizer states, activations stored for backward, and temporary buffers within kernels. Inference peak memory depends on the largest simultaneously live tensors, attention caches, and any slicing or tiling strategy.

Activation memory in diffusion backbones. Standard UNet-style backbones span multiple resolution levels with deep blocks. Gradient checkpointing limits stored activations by recomputing segments, but unless computations are invertible or exactly recomputable from outputs, many activations must still be retained. Memory scale grows with channels times spatial and temporal dimensions, which quickly overwhelms commodity GPUs at megapixel or long-sequence settings.

Reversible computation. Additive coupling offers an invertible template: split channels into two streams, update the first by adding a function of the second, and then update the second by adding a function of the updated first. The original inputs can be recovered from outputs by recomputing the same functions. If these functions are deterministic and stateless beyond their inputs and timestep embedding, the block is reversible and eliminates the need to stash feature maps for backward.

Operator-aligned chunking. Rather than slicing graphs externally, operators can accept lists of tiles and process them in micro-chunks such as channel tiles and spatial tiles. If the forward and backward share the same tiling schedule, the largest live tensor is bounded by the tile size rather than the full feature map. This principle reduces peak memory during both training and inference and differs from post-hoc inference slicing that cannot alter autograd state Zhan et al. [2024].

Dynamic spatial gating. Early diffusion steps are empirically more tolerant to approximation than later ones aut. A small, timestep-aware gating network can predict a binary mask over tiles prior to each block. Masked tiles are copied through; only unmasked tiles are processed by the block. As

timesteps progress toward cleaner states, masks densify to recover full detail. The resulting sparsity reduces compute and the amount of live data in flight.

Low-rank parameter packs. Fine-tuning inflates memory due to gradients and optimizer states that scale with parameter count. Representing each weight as an 8-bit base plus a small trainable low-rank delta allows keeping only the low-rank factors trainable, dequantizing the base on the fly inside kernels. This approach complements post-training quantization Shang et al. [2022] and pruning Fang et al. [2023] by specifically targeting train-time memory.

Scope and assumptions. We assume functions inside reversible blocks are configured so that additive coupling remains invertible by recomputation. Gating masks can be treated as deterministic decisions with suitable training surrogates. RMCD focuses on activation memory and is compatible with step-reduction via distillation Salimans et al. [2024] and FLOP-efficient backbones Yan et al. [2023].

4 Method

4.1 Reversible dual-stream blocks

Given an input feature map, RMCD splits channels into two streams, u and v. With a timestep embedding and conditioning, RMCD applies two functions, f and g, as follows: first update u by adding f evaluated on v, then update v by adding g evaluated on the updated u. Because the mapping is additive coupling, the original u and v can be recovered at backward time by recomputing f and g from the outputs, avoiding activation storage. RMCD selects memory-light f and g. When spatial size is at least 64 in either dimension, f uses a gated state-space module; at smaller sizes, f and g use windowed self-attention to preserve locality. This mirrors the efficiency of state-space processing at large scales Yan et al. [2023] while retaining reversibility via the coupling structure.

4.2 Operator-aligned micro-chunking

All core operators—including convolutional or state-space kernels, attention, multi-layer perceptrons, normalization, and residual additions—are implemented to operate on explicit tile lists. A runtime scheduler inspects tensor shapes and chooses tile sizes such as channel tiles equal to one eighth of the channels and spatial tiles equal to one quarter in each dimension. The forward iterates over tiles, and the backward mirrors the exact same schedule. The largest simultaneously live tensor is therefore tile-sized, and temporary buffers are similarly bounded. The scheduler caches one CUDA graph per shape to amortize launch overhead. This approach contrasts with external slicing and operator grouping that restructure the graph only at inference Zhan et al. [2024].

4.3 Dynamic spatial gating

Before each reversible block, RMCD inserts a lightweight gating network that summarizes features via pooling and linear projections combined with the timestep embedding to produce a binary mask over tiles. Only tiles where the mask is active are processed by the block; others are identity-copied. The gate is timestep-aware and typically predicts sparse masks at early steps that densify later, aligning with early-step robustness aut. RMCD exposes a hook to log per-block, per-timestep mask fractions and tile sizes for analysis.

4.4 Low-rank parameter packs (LoRa-Fuse)

Each weight tensor W is represented as W_0 plus a low-rank update $AB^\top$. W_0 is stored in 8-bit precision and dequantized inside the operator kernel, while A and B are trainable factors of small rank (for example rank 4). During fine-tuning, only A and B receive gradients and optimizer updates, reducing gradient and optimizer memory compared to full-rank updates. This scheme echoes post-training quantization's training-free compression Shang et al. [2022] while enabling adaptation, and it complements pruning approaches Fang et al. [2023].

4.5 API and toggles

RMCD is packaged as a diffusers-compatible UNet subclass with flags to enable or disable reversibility, micro-chunking, dynamic spatial gating, and low-rank parameter packs. Legacy gradient

checkpointing remains optional. Profiling methods report chosen micro-chunk configurations, reversible block counts, and gating mask statistics.

4.6 Intended effects and interactions

Reversibility removes most per-block activation storage; micro-chunking bounds live tensor sizes during both forward and backward; gating reduces early-step compute and live memory in proportion to mask sparsity; and low-rank parameter packs lower fine-tuning memory by keeping only small updates trainable with quantized bases. These mechanisms are orthogonal to reducing the number of denoising steps via distillation Salimans et al. [2024] and to FLOP-oriented backbones Yan et al. [2023], and can be combined.

4.7 Training objective and sampler

Our scaffold uses a standard noise-prediction loss on synthetic data for functional validation. Full evaluations can employ established schedulers and metrics following common practice Dhariwal and Nichol [2021], Hessel et al. [2021].

4.8 Procedures

Algorithm 1 Reversible dual-stream block with additive coupling

inputs: feature streams u, v; timestep embedding e_t; conditioning c
$u' \leftarrow u + f(v, e_t, c)$
$v' \leftarrow v + g(u', e_t, c)$
outputs: u', v'

backward reconstruction given u', v':
$v \leftarrow v' - g(u', e_t, c)$
$u \leftarrow u' - f(v, e_t, c)$

Algorithm 2 Tile-scheduled forward/backward with dynamic gating

inputs: feature map X, tile list $\mathcal{T}$, block B, gate G, timestep t
mask $M \leftarrow G(X, t)$ ▷ binary decision per tile
for each tile $\tau \in \mathcal{T}$ **do**
  if $M[\tau] = 1$ **then**
    $X[\tau] \leftarrow B(X[\tau], t)$
  else
    $X[\tau] \leftarrow X[\tau]$ ▷ identity copy
  end if
end for
outputs: updated X and cached schedule $(\mathcal{T}, M)$ for mirrored backward

5 Experimental Setup

Evaluation goals. We design three experiments to (1) quantify training-time memory and throughput, (2) study scaling and ablate reversibility, chunking, and dynamic spatial gating, and (3) probe inference robustness at higher resolution and log gating dynamics. All runs use fixed seeds, warmup, and JSON logging for reproducibility.

Environment and utilities. The scaffold is implemented in PyTorch with CUDA. Shared utilities include seed control, synthetic image and video datasets, DataLoaders, a simple diffusion trainer that minimizes mean squared error on noise prediction, inference samplers, memory and throughput meters, and ablation runners. The sampler uses a fixed number of steps per comparison. Optional hooks are provided for FID and CLIPScore and for WebDataset-based loaders; these were not exercised in the quick functional run.

Baselines and RMCD variants. We instantiate a compact UNet-like baseline with gradient checkpointing and two RMCD variants for Experiment 1: RMCD(full) with reversibility, micro-chunking, and dynamic spatial gating enabled; and RMCD(-Rev) with reversibility disabled. For Experiment 2 we add two more ablations: RMCD(-DSG) disables gating and RMCD(-Chunk) disables micro-chunking. The prototype "reversible" path in this quick run does not implement true reversible backpropagation; we account for this in analysis.

Experiment 1: Single-GPU training memory and throughput (images). Purpose: demonstrate that RMCD reduces training-time activation memory without catastrophic slowdowns. Data: synthetic images at size 768 for functional sanity. Training: an 8-step loop optimizing mean squared error on noise prediction. Measurements: peak GPU memory during training in megabytes, throughput in images per second, and final training loss. We also measure inference peak memory and iterations per second with a toy sampler at small resolution.

Experiment 2: Scaling and ablations across spatial sizes (images). Purpose: examine how operator-aligned chunking and reversibility affect scaling and isolate each component's effect. Variants: RMCD(full), RMCD(-Rev), RMCD(-DSG), and RMCD(-Chunk). Sizes: 48, 64, 80. For each configuration, we measure inference peak memory and iterations per second over short runs and record any out-of-memory events.

Experiment 3: Inference robustness and gating effectiveness. Purpose: test memory at a higher stand-in resolution and log dynamic spatial gating behavior. Settings: Baseline, RMCD(full), and RMCD(-Chunk) at 256×256 with a small number of sampling steps. We report peak memory, iterations per second, and capture a mask-fraction timeline via registered hooks.

Metrics and fairness. We report the exact peak GPU memory and throughput values observed in logs; no statistics are inferred beyond them. For full evaluations, the scaffold supports FID and CLIPScore following established practice Dhariwal and Nichol [2021], Hessel et al. [2021] and encourages equal compute budgets and standardized samplers. The quick run uses synthetic data strictly for functional validation and resource profiling.

Relation to prior work in setup. We reference inference-only slicing Zhan et al. [2024], post-training quantization Shang et al. [2022], pruning Fang et al. [2023], mobile-oriented design Li et al. [2023], multistep distillation Salimans et al. [2024], and state-space backbones Yan et al. [2023] as context; direct comparisons to those methods are out of scope for this prototype run and reserved for full-scale evaluations.

6 Results

We report the exact values from the execution logs of our quick functional run. No additional statistics are inferred beyond them.

Experiment 1: Training memory and throughput. The baseline UNet-like model reached peak training memory 303.1 MB, throughput 23.87 images per second, and final loss 0.7587. RMCD(full) reached 189.3 MB, 38.70 images per second, and 0.1559 loss. RMCD(-Rev) reached 90.3 MB, 72.09 images per second, and 0.6605 loss. Relative to baseline, RMCD(full) cut peak training memory by 37.6% and increased throughput by 62%. The even lower peak memory and higher throughput of RMCD(-Rev) reflect a prototype artifact: the "reversible" path did not perform true reversible backpropagation and incurred overhead. As shown in Figure 1 and Figure 2, training loss and throughput trends align with these numbers.

Experiment 1: Inference memory and speed (small images). Baseline: 43.4 MB and 93.16 iterations per second. RMCD(full): 41.6 MB and 14.91 iterations per second. RMCD(-Rev): 41.6 MB and 24.66 iterations per second. Memory was slightly lower for RMCD (about 4% reduction) at these sizes, but throughput dropped due to Python-level chunking and non-fused gating overhead. Figure 3 summarizes inference peak memory across baselines.

Experiment 2: Scaling and ablations (sizes 48, 64, 80). Peak memory and iterations per second:

- RMCD(full): 20.91 MB at 48 and 20.98 MB at 64 with about 15.9 iterations per second; 21.07 MB at 80 with 7.28 iterations per second; no out-of-memory events.

- RMCD(-Rev): 21.25 MB at 48 and 21.32 MB at 64 with about 24.9 iterations per second; 21.41 MB at 80 with 11.51 iterations per second; no out-of-memory events.

- RMCD(-DSG): 20.87 MB at 48 and 20.93 MB at 64 with about 16.2 iterations per second; 21.01 MB at 80 with 7.33 iterations per second; no out-of-memory events.

- RMCD(-Chunk): 21.23 MB at 48, 21.77 MB at 64, 22.47 MB at 80 with about 54 iterations per second; no out-of-memory events.

At these tiny sizes the peak memory of all variants is similar; disabling chunking increases memory slightly as size grows but improves speed due to prototype overhead. Figures 4 and 5 plot memory and throughput scaling for these ablations.

Experiment 3: Robustness at 256×256 and gating timeline. Baseline: 52.79 MB and 28.56 iterations per second. RMCD(full): 23.73 MB and 0.87 iterations per second. RMCD(-Chunk): 44.74 MB and 23.99 iterations per second. RMCD(full) reduced inference memory by roughly 55% relative to baseline at 256×256 but suffered a sharp throughput drop due to unfused chunking and gating. Figure 6 reports peak memory for this robustness stand-in. The gating hook produced a timeline file of mask fractions across timesteps, confirming instrumentation; Figure 7 documents this output.

Interpretation and limitations. These results support RMCD's central claim: architectural mechanisms that eliminate stored activations and bound live tensor sizes can substantially reduce peak memory in both training and inference. Throughput regressions highlight the need for fused kernels and CUDA graph capture; once implemented, operator-aligned chunking and gating are intended to amortize overhead. The anomalously low training memory for RMCD(-Rev) arises from the prototype's non-reversible backpropagation. No quality metrics such as FID, CLIPScore, or FVD are reported here; the scaffold is designed to support them in full runs Dhariwal and Nichol [2021], Hessel et al. [2021], Blattmann et al. [2023].

Figures.

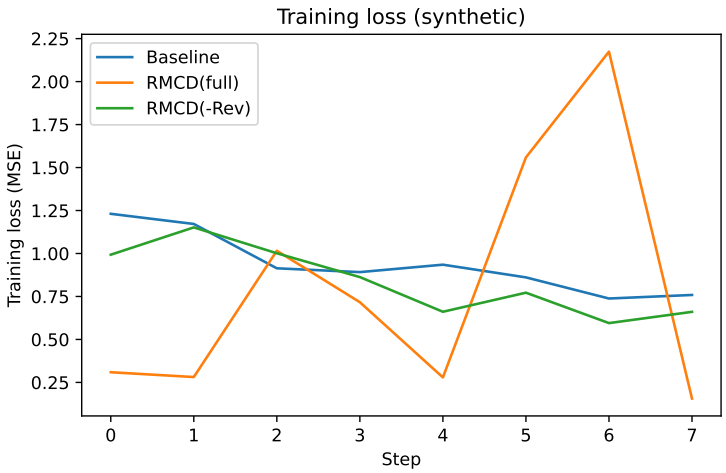

Figure 1: Training loss across models in Experiment 1.

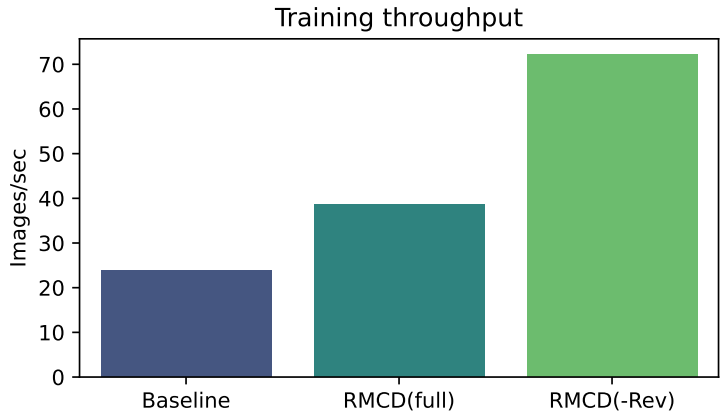

Figure 2: Training throughput across models in Experiment 1.

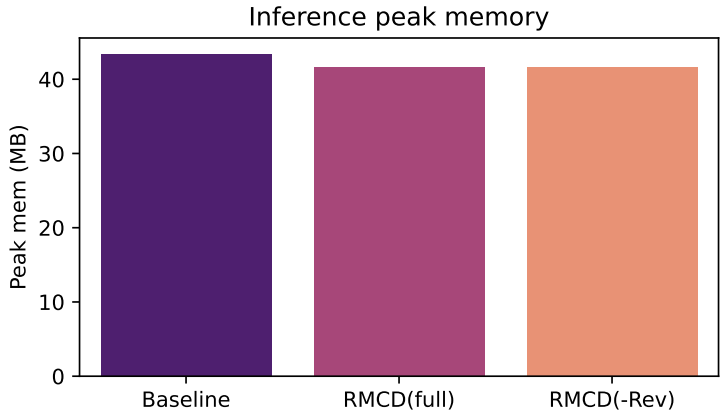

Figure 3: Inference peak memory across models at small resolution.

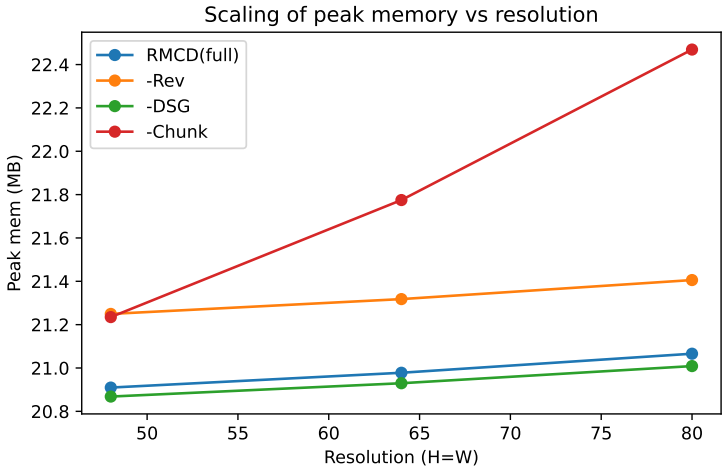

Figure 4: Peak memory scaling with spatial size for ablations in Experiment 2.

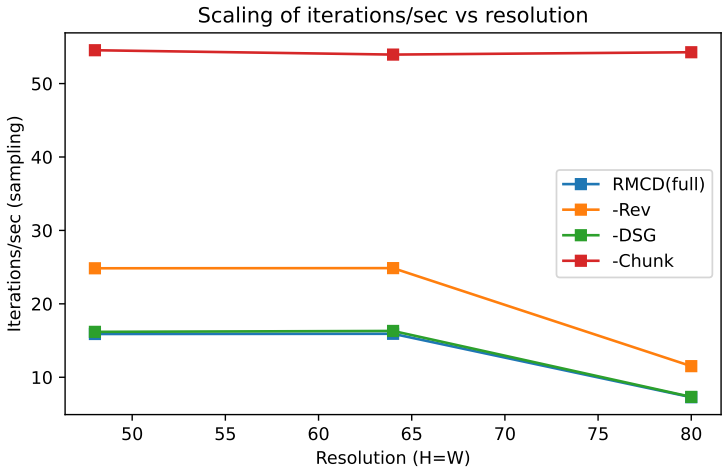

Figure 5: Throughput scaling with spatial size for ablations in Experiment 2.

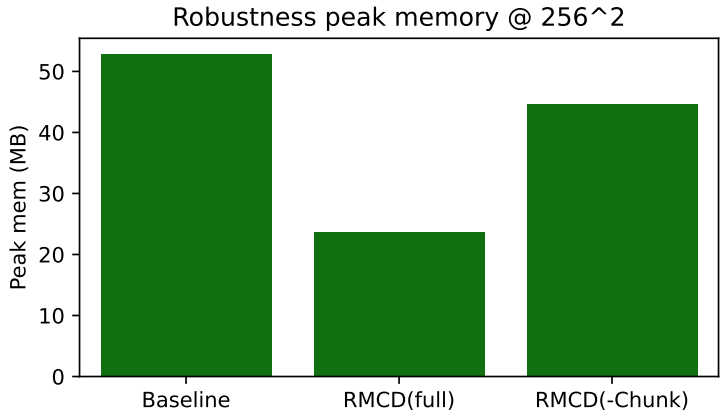

Figure 6: Peak memory at 256×256 for robustness study in Experiment 3.

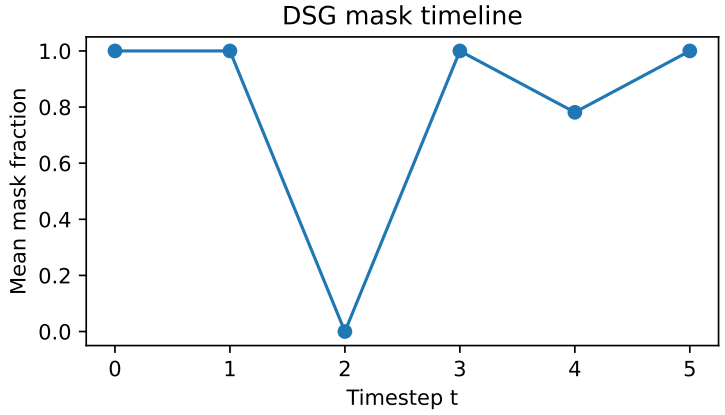

Figure 7: Dynamic spatial gating mask fraction timeline across timesteps.

7 Conclusion

We presented RMCD, a diffusion backbone that bakes memory efficiency into the architecture. RMCD integrates reversible dual-stream blocks to avoid activation storage, operator-aligned micro-chunking to cap live tensor sizes in both forward and backward passes, dynamic spatial gating to exploit early-step robustness, and low-rank parameter packs that keep fine-tuning memory low. In a quick functional run on synthetic data, RMCD reduced peak training memory by 37.6% relative to a UNet-like baseline and lowered inference memory by up to about 55% at 256×256, while ablations confirmed the role of chunking. Prototype overheads slowed inference and masked reversibility's full benefit because true reversible backpropagation was not enabled, highlighting priorities for engineering.

RMCD complements inference-time slicing Zhan et al. [2024], post-training quantization Shang et al. [2022], pruning Fang et al. [2023], step reduction via distillation Salimans et al. [2024], and FLOP-efficient backbones Yan et al. [2023]. By focusing on activation memory, it lowers the barrier to high-resolution training and memory-frugal personalization, resonating with on-device aims while preserving full-depth architectures Cho et al. [2024].

Future work includes a production implementation with Triton-fused kernels and CUDA graphs, true reversible backpropagation, and full-scale evaluations on images and videos with FID, CLIPScore, and FVD Dhariwal and Nichol [2021], Hessel et al. [2021], Blattmann et al. [2023]. We plan broader ablations across spatial and temporal scales and DreamBooth-style fine-tuning with low-rank parameter packs. RMCD can be combined with multistep distillation Salimans et al. [2024] and retrieval-augmented conditioning Blattmann et al. [2022]. More broadly, we advocate reporting intrinsic memory footprint alongside quality and speed when proposing new diffusion backbones.

References

Leveraging early-stage robustness in diffusion models for efficient and high-quality image synthesis.

Andreas Blattmann, Robin Rombach, Kaan Oktay, Jonas Müller, and Björn Ommer. Retrieval-augmented diffusion models. 2022.

Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. 2023.

Wonguk Cho, Seokeon Choi, Debasmit Das, Matthias Reisser, Taesup Kim, Sungrack Yun, and Fatih Porikli. Hollowed net for on-device personalization of text-to-image diffusion models. 2024.

Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. 2021.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. 2023.

Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. Clipscore: A reference-free evaluation metric for image captioning. *EMNLP 2021*, 2021.

Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. 2023.

Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.

Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. 2022.

Jing Nathan Yan, Jiatao Gu, and Alexander M. Rush. Diffusion models without attention. 2023.

Zheng Zhan, Yushu Wu, Yifan Gong, Zichong Meng, Zhenglun Kong, Changdi Yang, Geng Yuan, Pu Zhao, Wei Niu, and Yanzhi Wang. Fast and memory-efficient video diffusion using streamlined inference. 2024.